

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ДНІПРОВСЬКА ПОЛІТЕХНІКА»

Факультет: Інформаційних технологій

Кафедра: Програмного забезпечення комп'ютерних систем

ЗВІТ

з дисципліни: ПРАКТИКА НАВЧАЛЬНА КОМП'ЮТЕРНА

Студентки 3 курсу, групи 121-23з-1

Спеціальності Інженерія ПЗ

Марковська Д. С.

Керівник асистент Харь А.Т.

Національна шкала _____

Кількість балів: ____ Оцінка: ECTS ____

(підпис) Харь А.Т.
(прізвище та ініціали)

м. Дніпро – 2025 рік

Зміст

Зміст.....	2
Завдання на практику.....	3
Створення концептуальної моделі предметної області.....	4
Створення UML-діаграми класів.....	8
Створення Deployment diagram.....	9
Налаштування адміністративної панелі Django.....	11
Реалізація логіки відображення та експорту даних у файлі views.py.....	12
Створення інтерфейсу користувача.....	14
Висновки.....	15
Додатки.....	18
Додаток 1.....	18
Додаток 2.....	19
Додаток 3.....	22

Завдання на практику

Навчальна практика виконана на базі розробок, реалізованих у рамках курсової з дисципліни «Організація баз даних та знань».

Розробити інформаційно-пошукову модель бази даних та програмного забезпечення, пов'язаного з нею, для працівників онлайн магазину жіночого одягу, що працює по системі дропшипінг. Створена програма в поєднанні з базою даних надасть працівнику магазину можливість перевірити наявність товару за його артикулом, знайти товари за потрібним кольором, розміром чи категорією, знайти заміри потрібної речі за артикулом моделі та зв'язатися з постачальником для оформлення замовлення.

Створення концептуальної моделі предметної області

Важливу роль в розробці програмних засобів займає аналіз предметної області, проектування системи, розробка схеми бази даних. Від цього етапу розробки залежить швидкодія застосування та коректність виведення результатів.

Для коректної роботи програми відповідно до завдання, створена база даних має містити сутності та зв'язки між ними, наведені нижче.

Сутності

1. Продукти (Products_OD):
 - Артикул (Code)
 - Назва (Product_Name)
 - Опис (Description)
 - Категорія (Categories_OD_Category_Name)
 - Зображення (Image_URL)
 - Ціна (Price)
2. Категорії (Categories_OD):
 - Назва категорії (Category_Name)
3. Кольори (Colors_OD):
 - Назва кольору (ColorName)
4. Розміри (Sizes_OD):
 - Ідентифікатор розміру (Size_ID)
 - Назва розміру (Size_Name)
5. Заміри (Measures_OD):
 - Ідентифікатор замірів (Measure_ID)
 - Артикул (Products_OD_Code)
 - Фото замірів (Img_URL)
 - Опис (Description)
6. Постачальники (Suppliers_OD):

- Код постачальника (Supplier_ID)
 - Назва постачальника (Supplier_Name)
 - Посилання (Supplier_URL)
7. Інвентаризація (Inventory_OD):
- Ідентифікатор обліку (Inventory_ID)
 - Артикул (Products_OD_Code)
 - Ідентифікатор розміру (Sizes_OD_Size_ID)
 - Назва кольору (Colors_OD_Color_Name)
 - Наявність на складі (Available)
 - Bltynsabrfnjh gjcnfxfkmysrf (Suppliers_OD_Supplier_ID)

Зв'язки

Продукт -> Категорія (багато-до-одного): Кожен продукт належить до певної категорії.

Продукт -> Інвентар (один-до-багатьох): Кожен продукт має запис у таблиці інвентарю з різними розмірами, кольорами та наявністю.

Продукт -> Заміри (один-до-одного): Кожному продукту відповідає один замір.

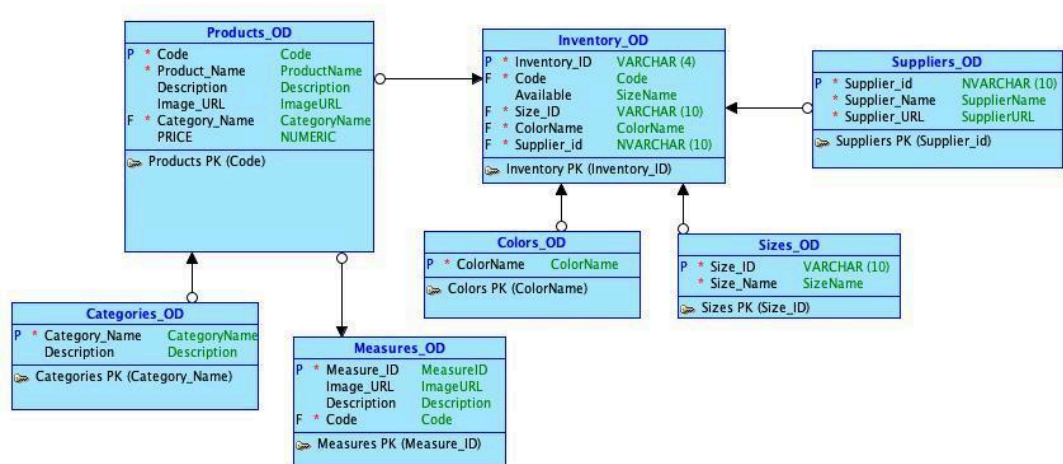
Інвентар -> Розміри (багато-до-одного): Інвентар має інформацію про доступні розміри для продукту.

Інвентар -> Кольори (багато-до-одного): Інвентар також містить інформацію про доступні кольори.

Інвентар -> Постачальник (багато-до-одного): Постачальники надають певні продукти з різними розмірами та кольорами.

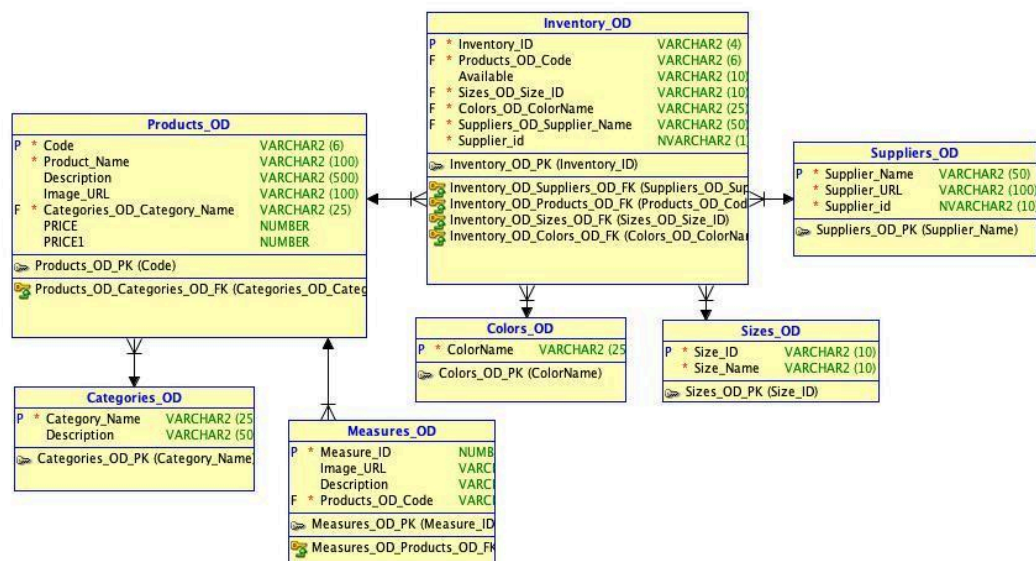
На основі цих даних проектуємо логічну модель майбутньої бази даних за допомогою OracleDataModeler, хоча надалі у цій роботі буде використовуватись

інша СУБД, цей додаток допоможе отримати готовий SQL запит для створення бази даних, який потрібно буде лише трохи змінити, бо у Oracle та PostgreSQL відрізняються назви типів даних.



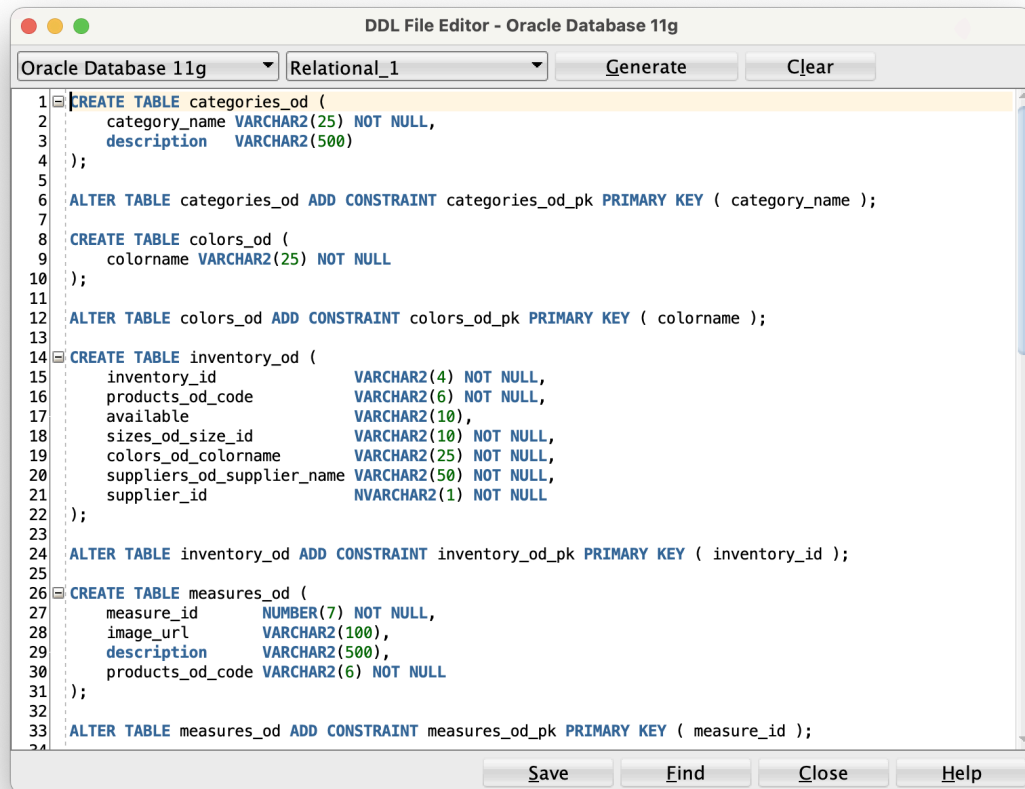
Мал. 1 Логічна модель бази даних

Після створення логічної моделі бази даних, можна досить просто конвертувати ці дані для створення ER-діаграми, у програмі OracleDataModeler. Після чого отримаємо модель бази даних, як показано на малюнку 2.



Мал. 2 ER-діаграма бази даних

Всі потрібні кроки для створення бази даних зроблено, тепер експортуємо дані у DDL файл, перед цим обираємо потрібну версію Oracle, від цього залежить синтаксис запитів, які генерує програма. Отримуємо готові SQL запити, які створять всі необхідні таблиці та зв'язки між ними.



```
DDL File Editor - Oracle Database 11g
Oracle Database 11g Relational_1 Generate Clear

1 CREATE TABLE categories_od (
2     category_name VARCHAR2(25) NOT NULL,
3     description   VARCHAR2(500)
4 );
5
6 ALTER TABLE categories_od ADD CONSTRAINT categories_od_pk PRIMARY KEY ( category_name );
7
8 CREATE TABLE colors_od (
9     colorname VARCHAR2(25) NOT NULL
10 );
11
12 ALTER TABLE colors_od ADD CONSTRAINT colors_od_pk PRIMARY KEY ( colorname );
13
14 CREATE TABLE inventory_od (
15     inventory_id      VARCHAR2(4) NOT NULL,
16     products_od_code  VARCHAR2(6) NOT NULL,
17     available         VARCHAR2(10),
18     sizes_od_size_id  VARCHAR2(10) NOT NULL,
19     colors_od_colorname VARCHAR2(25) NOT NULL,
20     suppliers_od_supplier_name VARCHAR2(50) NOT NULL,
21     supplier_id       NVARCHAR2(1) NOT NULL
22 );
23
24 ALTER TABLE inventory_od ADD CONSTRAINT inventory_od_pk PRIMARY KEY ( inventory_id );
25
26 CREATE TABLE measures_od (
27     measure_id      NUMBER(7) NOT NULL,
28     image_url       VARCHAR2(100),
29     description     VARCHAR2(500),
30     products_od_code VARCHAR2(6) NOT NULL
31 );
32
33 ALTER TABLE measures_od ADD CONSTRAINT measures_od_pk PRIMARY KEY ( measure_id );
34

Save Find Close Help
```

Мал. 3 Згенерований запит для створення БД за ER-діаграмою

Так як цей запит створено під СУБД Oracle SQLDeveloper, а ми плануємо працювати на PostgreSQL у СУБДР TablePlus, маємо змінити типи даних:

- NVARCHAR2 та VARCHAR2 на VARCHAR;
- NUMBER на NUMERIC

Відредаговані запити зберігаємо у SQL файл та створюємо за допомогою нього базу даних з усіма таблицями. Повний код для створення таблиць можна переглянути у додатку 1 «Запит для створення БД».

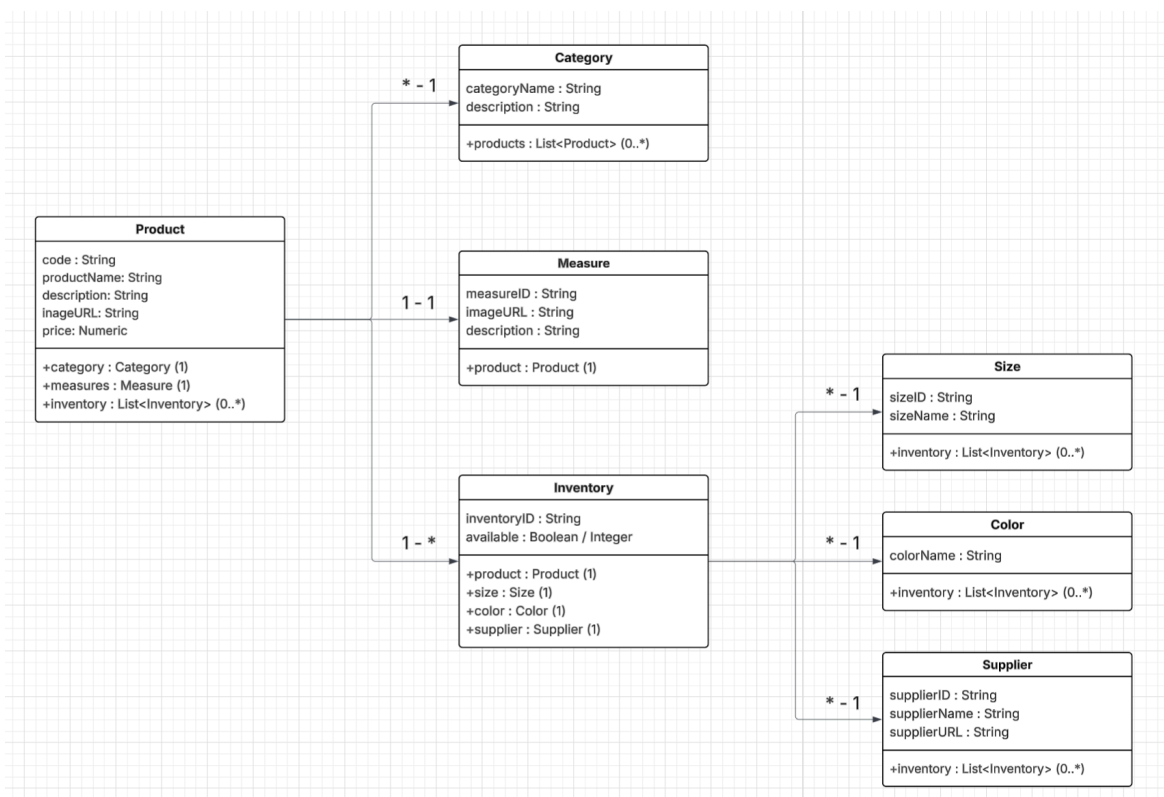
Створення UML-діаграми класів

UML дозволяє перейти від поняття "таблиця" до "класу", що використовується у програмуванні об'єктно-орієнтованих мов.

Основні кроки:

1. **Кожна таблиця БД → клас UML.** Наприклад, таблиця Products_OD відповідає класу Product.
2. **Атрибути таблиць → атрибути класів.** Поля Code, Product_Name, Price стали атрибутами класу Product.
3. **Зовнішні ключі → асоціації між класами.** Зв'язок "Product – Category" відображається як асоціація з множинністю $* \rightarrow 1$.
4. **Кардинальності у ER → множинності у UML.**
 - Product має багато Inventory ($1 \rightarrow *$).
 - Inventory належить одному Supplier ($* \rightarrow 1$).
 - Product має один Measure ($1 \leftrightarrow 1$).

Для побудови UML-діаграми було використано онлайн ресурси.



Мал. 3 UML-діаграма класів

https://lucid.app/lucidchart/166bc2d7-00fc-4e68-8649-d1379991286e/edit?invitationId=inv_7029320e-87f5-4949-9b59-6820e784d56c

Створення Deployment diagram

Для відображення архітектури веб-застосунку була створена **діаграма розгортання (Deployment Diagram)** у нотації UML. Вона демонструє фізичне розміщення основних компонентів системи та взаємозв'язки між ними.

На діаграмі виділено чотири основні вузли.

- **Client (клієнт)** – веб-браузер, що відображає інтерфейс сайту та дозволяє користувачеві виконувати запити за допомогою HTML, CSS і JavaScript.
- **Web Server (веб-сервер)** – сервер із встановленим Apache, який приймає HTTP-запити від клієнта та передає їх на сервер застосунку.
- **Application Server (сервер застосунку)** – реалізований на Django/PHP, відповідає за бізнес-логіку: пошук і відбір товарів, перевірку наявності, роботу з категоріями, кольорами, розмірами та постачальниками.
- **Database Server (сервер бази даних)** – PostgreSQL, що зберігає усі дані про товари, категорії, характеристики та постачальників.

Зв'язки між вузлами реалізовані у вигляді послідовної взаємодії: клієнт надсилає запити веб-серверу, веб-сервер передає їх на сервер застосунку, який у свою чергу звертається до бази даних для отримання або збереження інформації. Така архітектура забезпечує розподіл функціональних обов'язків і підвищує гнучкість та масштабованість системи.



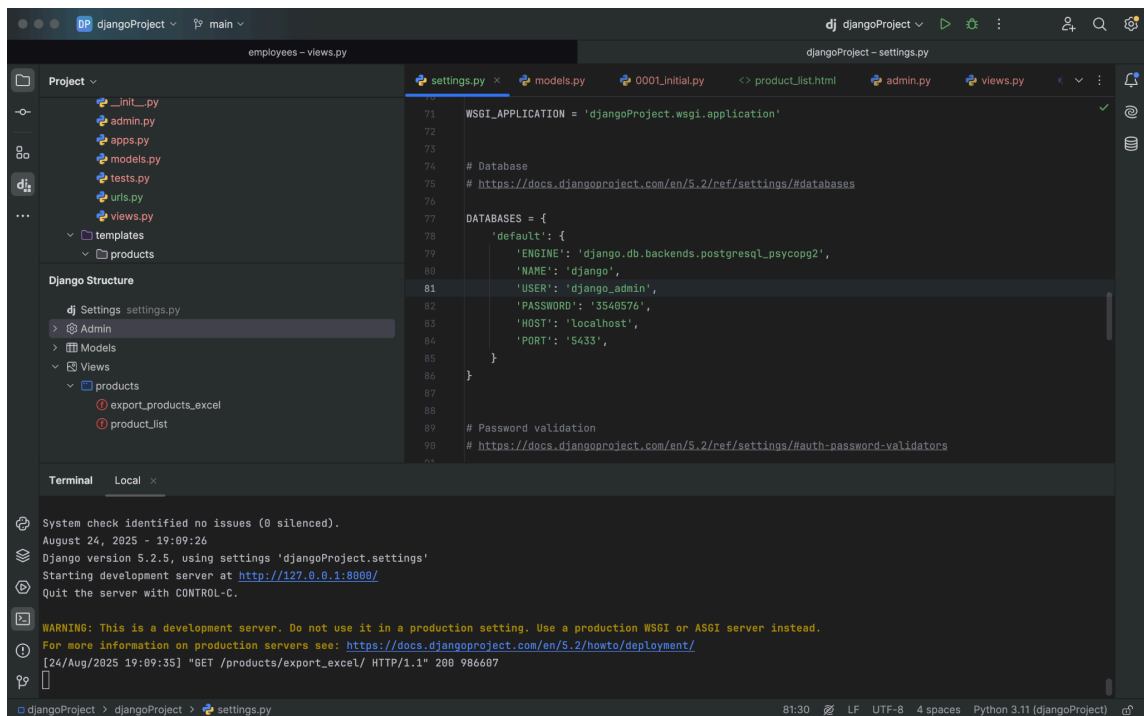
Мал. 4 Deployment diagram

https://lucid.app/lucidchart/9475b7af-9ec6-41b1-99dc-784ec9fa6b20/edit?view_items=Af.HZLWlcm9O&invitationId=inv_6f7ea100-e746-40b6-8c80-ea1c01210d60

Створення веб-серверного застосунку

Для створення застосунку на базі python django налаштовуємо базу даних - створюємо нову БД для роботи з django, створюємо користувача та надаємо йому

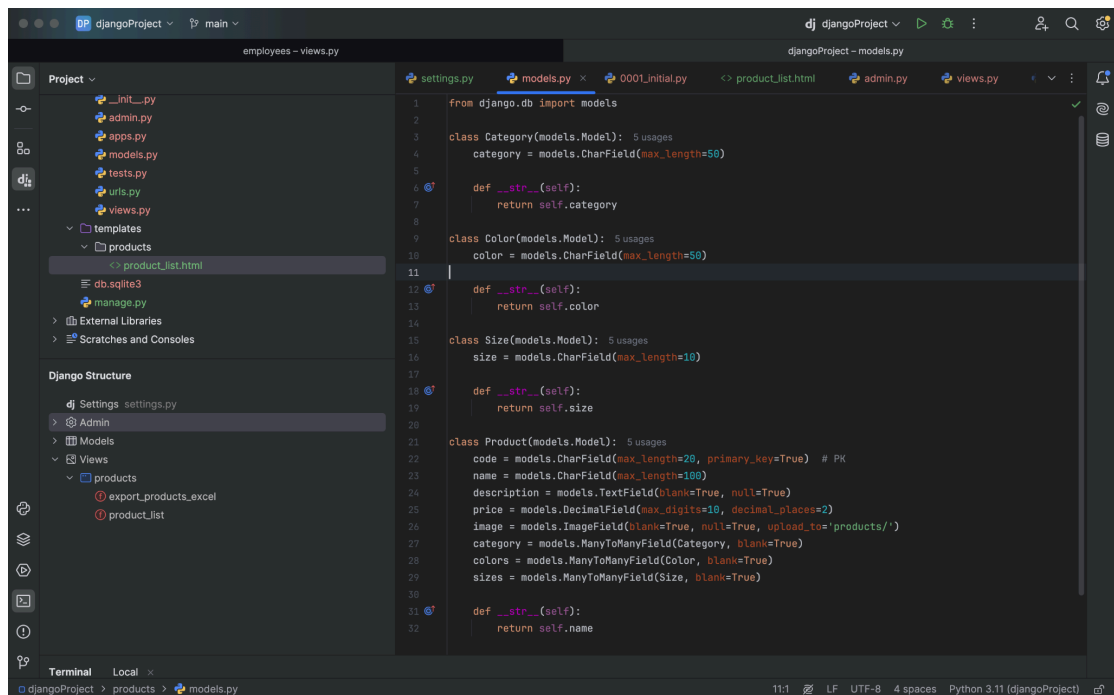
потрібні для коректної роботи django migrations дозволи, після чого вносимо зміни до файлу settings.py та підключаємо її у проєкті.



Мал. 5 Підключення до БД

Для роботи в Django з базою даних PostgreSQL використовуються models описуємо їх для створення всіх таблиць, потрібних для створення БД описаній в п.1.1.

Скрипт Файлу [models.py](#) можна побачити в додаток 1.



Мал. 6 Моделі

Для створення таблиць прописаних у моделях застосовуємо міграції django, за допомогою команд:

```
python manage.py makemigrations  
python manage.py migrate
```

Налаштування адміністративної панелі Django

Було здійснено налаштування адміністративної панелі веб-застосунку за допомогою вбудованого інструменту **Django Admin**. Це дозволило забезпечити зручне керування довідниками та основними сутностями бази даних інтернет-магазину одягу.

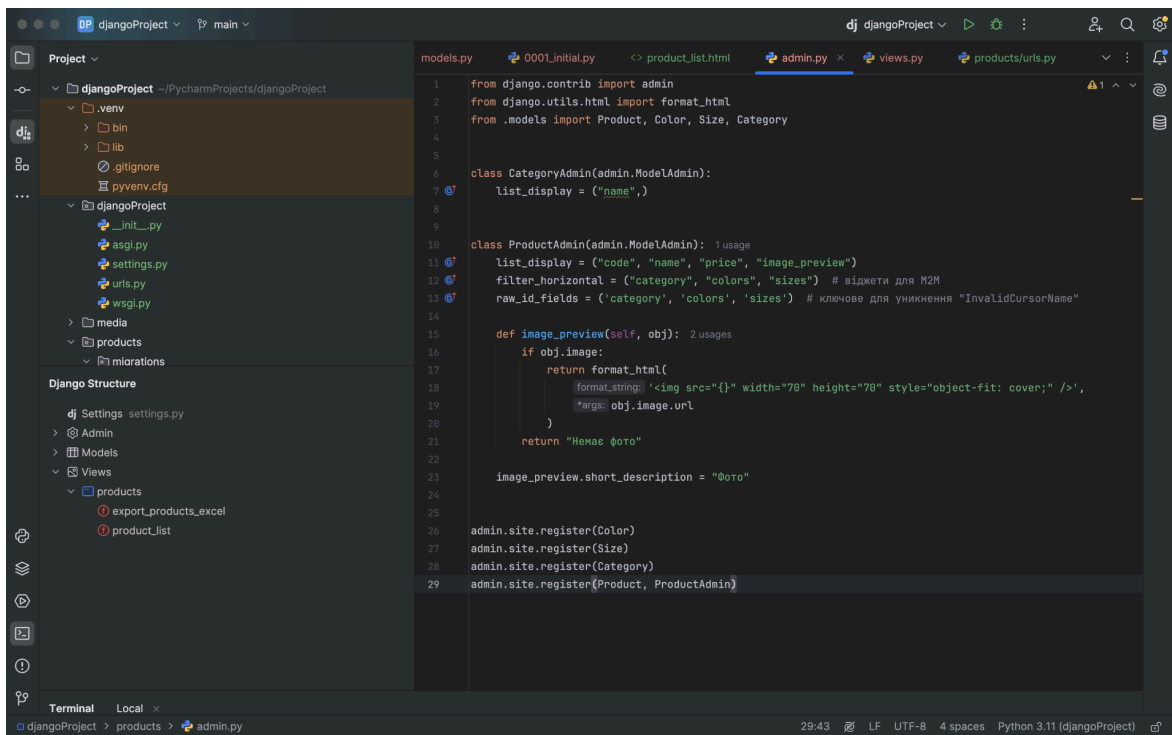
Для відображення категорій було створено клас `CategoryAdmin`, у якому визначено поле `list_display`. Завдяки цьому адміністратор отримує можливість переглядати список категорій у табличному вигляді з відображенням їх назв.

Для моделі товарів (`Product`) створено спеціальний клас `ProductAdmin`, у якому передбачено низку розширених можливостей:

- у полі `list_display` визначено виведення коду, назви, ціни та мініатюри зображення товару;
- для зв'язків «багато-до-багатьох» (категорії, кольори, розміри) використано параметр `filter_horizontal`, що дозволяє зручно додавати або вилучати пов'язані записи;
- застосовано параметр `raw_id_fields`, який забезпечує роботу з великою кількістю даних без перевантаження інтерфейсу та попереджає помилки типу *InvalidCursorName*.

Окремо реалізовано метод `image_preview`, що дає змогу переглядати мініатюру фотографії товару безпосередньо в таблиці адміністративної панелі. Якщо зображення відсутнє, замість нього виводиться відповідний текст «Немає фото». Це значно спрощує роботу адміністратора з великим асортиментом продукції.

У підсумку, у адміністративній панелі було зареєстровано всі основні моделі системи (`Color`, `Size`, `Category`, `Product`), що надало можливість здійснювати повноцінне керування даними в інтуїтивно зрозумілому інтерфейсі.



Мал. 7 Файл admin.py

Реалізація логіки відображення та експорту даних у файлі views.py

У ході практики було реалізовано функціонал відображення списку товарів із можливістю фільтрації, а також експорт цих даних до Excel-файлу. Логіка роботи веб-застосунку була зосереджена у файлі **views.py**, що відповідає за обробку HTTP-запитів та формування відповідей.

1. Функція `export_products_excel`

Дана функція реалізує **експорт даних про товари до файлу Excel** з використанням бібліотеки **OpenPyXL**.

Основні етапи роботи:

- створення нового Excel-файлу (Workbook) та активного аркуша з назвою *Products*;
- формування заголовків таблиці (код, назва, опис, ціна, зображення, категорії, кольори, розміри);
- послідовне проходження всіх товарів у базі даних (`Product.objects.all()`);

- для зв'язків типу *багато-до-багатьох* (категорії, кольори, розміри) здійснюється злиття значень у рядки;
- у таблицю додаються текстові дані (код, назва, опис, ціна тощо) з форматкуванням — перенесення тексту (wrapText) та вертикальне вирівнювання;
- окремо реалізовано **додавання зображення** для кожного товару у вигляді мініатюри в комірку стовпця *E*. Виконується масштабування зображення із збереженням пропорцій та встановленням висоти рядка;
- встановлюється оптимальна ширина колонок (для опису ширша, для коду та ціни вужча);
- у кінці формується HTTP-відповідь зі згенерованим файлом, який автоматично завантажується користувачем під назвою *products.xlsx*.

Цей функціонал дозволяє адміністратору або користувачу експортувати повну базу товарів у зручному для подальшої обробки форматі Excel.

2. Функція `product_list`

Ця функція відповідає за **відображення списку товарів на сторінці каталогу** з можливістю застосування фільтрів.

Алгоритм роботи:

- отримуються всі товари (`Product.objects.all()`), а також списки категорій, кольорів та розмірів для формування елементів фільтрації на сторінці;
- перевіряються GET-параметри, передані користувачем (категорії, кольори, розміри, мінімальна та максимальна ціна);
- у випадку наявності фільтрів формується відповідний запит до бази даних (наприклад, `products.filter(category__id__in=category_ids)`);
- використовується метод `distinct()`, щоб уникнути дублювання товарів при фільтрації через зв'язки *багато-до-багатьох*;
- у словнику `context` формуються дані для передачі у шаблон (`products/product_list.html`), де відбувається відображення товарів та панелі фільтрації.

Таким чином, користувач отримує гнучкий інструмент для пошуку потрібних товарів за категоріями, кольорами, розмірами та ціновим діапазоном.

Повний лістинг коду можна переглянути у Додаток 2.

Створення інтерфейсу користувача

У межах практики було розроблено веб-інтерфейс для відображення списку товарів та взаємодії користувача з каталогом інтернет-магазину. Для створення інтерфейсу використано **HTML5**, **шаблонізатор Django** (конструкції `{% ... %}`, `{{ ... }}`), а також CSS-бібліотеку **Bootstrap 4**, яка забезпечує адаптивність і зручний дизайн.

1. Загальна структура

- У `<head>` підключено Bootstrap для стилізації елементів інтерфейсу.
- У `<body>` розміщено навігаційну панель із кнопками та основний блок зі списком товарів.
- Для інтерактивності застосовано JavaScript: реалізовано відкриття/закриття панелі фільтрації.

2. Навігаційна панель

У верхній частині сторінки створено **панель навігації (navbar)**, що включає:

- кнопку «Додати товар» (призначена для переходу до форми створення нового запису);
- кнопку «Фільтрувати товари», яка відкриває бокову панель із параметрами пошуку;
- кнопку «Експортувати в Excel», що дозволяє завантажити файл з усіма товарами (використовує функцію `export_products_excel` з `views.py`);
- заголовок сторінки «Список товарів».

3. Бокове меню фільтрації

Реалізовано інтерактивне **бокове меню**, яке з'являється праворуч після натискання кнопки «Фільтрувати товари». У ньому користувач може обрати:

- одну або кілька **категорій**;
- кольори товарів;
- розміри;
- ціновий діапазон (мінімальна та максимальна ціна).

Форма надсилається методом **GET** на адресу `product_list`, що забезпечує динамічну фільтрацію каталогу.

4. Відображення списку товарів

У головному контейнері (`<div class="container">`) реалізовано відображення товарів у вигляді **карток (card)**.

- Для кожного товару перевіряється наявність зображення (`{% if product.image %}`):
 - якщо фото є — воно відображається зверху картки;
 - якщо фото відсутнє — виводиться заглушка «Немає фото».
- У тілі картки (`card-body`) показано код товару, його назву та ціну.
- Завдяки використанню сітки Bootstrap (`row, col-md-3`) картки адаптуються під різні розміри екрану, що забезпечує зручність перегляду на комп'ютерах і мобільних пристроях.

5. Підключення скриптів

У кінці документа підключено бібліотеки:

- **jQuery**, **Popper.js** та **Bootstrap.js** для роботи компонентів інтерфейсу;
- власний JavaScript-код для керування відображенням бокового меню фільтрів:
 - відкриття при натисканні кнопки «Фільтрувати товари»;
 - закриття при натисканні кнопки «X»;
 - автоматичне закриття при кліку поза областю меню.

Повний скрипт коду можна побачити у Додаток 3.

Висновки

У результаті проходження навчальної практики було виконано повний цикл створення веб-застосунку інтернет-магазину одягу. Робота над проектом передбачала як теоретичне закріплення знань, отриманих під час навчання, так і практичну реалізацію завдань, пов'язаних із сучасними веб-технологіями.

На першому етапі було спроектовано базу даних, що забезпечує зберігання інформації про товари, категорії, кольори, розміри та інші характеристики. Для цього використано СУБД PostgreSQL, яка дозволяє надійно організувати роботу з даними та підтримує складні зв'язки між таблицями. Створена структура бази даних забезпечує масштабованість та можливість подальшого розширення функціональності застосунку.

Наступним етапом стала реалізація серверної частини на основі фреймворку Django. Було налаштовано адміністративну панель для управління товарами, категоріями та характеристиками продукції. Також реалізовано відображення списку товарів, можливість застосування фільтрів за кількома параметрами (категорія, колір, розмір, ціна), а також функцію експорту даних у формат Excel. У процесі розробки вдалося відпрацювати роботу з ORM, Many-to-Many зв'язками та спеціалізованими віджетами адміністративного інтерфейсу.

Окрему увагу приділено створенню інтерфейсу користувача. За допомогою HTML, CSS і Bootstrap було розроблено зручний веб-інтерфейс, що забезпечує адаптивність під різні пристрої. Було реалізовано відображення товарів у вигляді карток із фотографіями, описом і ціною, а також інтерактивне бічне меню для застосування фільтрів. Використання JavaScript дало змогу додати динамічні елементи, зокрема відкриття та закриття меню фільтрації.

Важливим аспектом виконаної роботи стало ознайомлення з архітектурою веб-застосунків. У ході практики було розроблено deployment diagram, що ілюструє взаємодію між клієнтом, веб-сервером, сервером застосунків та сервером бази даних. Це дозволило наочно показати розподіл функцій між компонентами системи та краще зрозуміти механізми їх взаємодії.

Таким чином, у процесі проходження практики було отримано цінний досвід у проектуванні, реалізації та документуванні веб-застосунків. Виконана робота дозволила закріпити знання з таких дисциплін, як веб-програмування,

робота з базами даних, серверні технології та системне проектування. Отримані результати підтверджують, що комплексний підхід до розробки програмних систем є ефективним шляхом до створення сучасних та функціональних веб-рішень.

Додатки

Додаток 1

```
from django.db import models

class Category(models.Model):
    category = models.CharField(max_length=50)

    def __str__(self):
        return self.category

class Color(models.Model):
    color = models.CharField(max_length=50)

    def __str__(self):
        return self.color

class Size(models.Model):
    size = models.CharField(max_length=10)

    def __str__(self):
        return self.size

class Product(models.Model):
    code = models.CharField(max_length=20, primary_key=True) # PK
    name = models.CharField(max_length=100)
    description = models.TextField(blank=True, null=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    image = models.ImageField(blank=True, null=True, upload_to='products/')
    category = models.ManyToManyField(Category, blank=True)
    colors = models.ManyToManyField(Color, blank=True)
    sizes = models.ManyToManyField(Size, blank=True)

    def __str__(self):
        return self.name
```

Додаток 2

```
from django.shortcuts import render
from django.http import HttpResponse
from openpyxl import Workbook
from .models import Product, Category, Size, Color

from openpyxl.drawing.image import Image as XLImage
from openpyxl.styles import Alignment
import os
from django.conf import settings

def export_products_excel(request):
    wb = Workbook()
    ws = wb.active
    ws.title = "Products"

    # Заголовки
    headers = ["Code", "Name", "Description", "Price", "Image", "Categories",
"Colors", "Sizes"]
    ws.append(headers)

    row_height = 200
    ws.row_dimensions[1].height = row_height # заголовков

    row_num = 2
    for p in Product.objects.all():
        # ManyToMany поля
        categories = ", ".join([c.category for c in p.category.all()])
        colors = ", ".join([c.color for c in p.colors.all()])
        sizes = ", ".join([s.size for s in p.sizes.all()])

        # Додаємо текстові дані
        ws.cell(row=row_num, column=1, value=p.code)
        ws.cell(row=row_num, column=2, value=p.name)
        ws.cell(row=row_num, column=3, value=p.description)
        ws.cell(row=row_num, column=4, value=float(p.price))
        ws.cell(row=row_num, column=6, value=categories)
        ws.cell(row=row_num, column=7, value=colors)
        ws.cell(row=row_num, column=8, value=sizes)
```

```

        # Wrap text та вертикальне вирівнювання для всіх текстових колонок
        for col in ["A", "B", "C", "D", "F", "G", "H"]:
            ws[col + str(row_num)].alignment = Alignment(wrapText=True,
vertical='center')

        # Додаємо фото у колонку E
        if p.image:
            image_path = os.path.join(settings.MEDIA_ROOT, p.image.name)
            if os.path.exists(image_path):
                img = XLImage(image_path)
                # Масштабування по висоті рядка з збереженням пропорцій
                aspect_ratio = img.width / img.height
                img.height = row_height
                img.width = int(row_height * aspect_ratio)
                ws.add_image(img, f"E{row_num}")

        # Встановлюємо висоту рядка
        ws.row_dimensions[row_num].height = row_height
        row_num += 1

    # Налаштування ширини колонок
    ws.column_dimensions["A"].width = 15
    ws.column_dimensions["B"].width = 25
    ws.column_dimensions["C"].width = 60    # опис
    ws.column_dimensions["D"].width = 12
    ws.column_dimensions["E"].width = 25    # картинка
    ws.column_dimensions["F"].width = 25
    ws.column_dimensions["G"].width = 25
    ws.column_dimensions["H"].width = 25

    response = HttpResponse(

content_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
    )
    response['Content-Disposition'] = 'attachment; filename="products.xlsx"'
    wb.save(response)
    return response

def product_list(request):
    # Отримуємо всі товари
    products = Product.objects.all().distinct()

```

```

# Отримуємо списки для фільтрів
categories = Category.objects.all()
colors = Color.objects.all()
sizes = Size.objects.all()

# Перевіряємо GET-параметри для фільтру
category_ids = request.GET.getlist('category')
color_ids = request.GET.getlist('color')
size_ids = request.GET.getlist('size')
min_price = request.GET.get('min_price')
max_price = request.GET.get('max_price')

# Фільтруємо лише якщо користувач обрав параметри
if category_ids:
    products = products.filter(category__id__in=category_ids)
if color_ids:
    products = products.filter(colors__id__in=color_ids)
if size_ids:
    products = products.filter(sizes__id__in=size_ids)
if min_price:
    products = products.filter(price__gte=min_price)
if max_price:
    products = products.filter(price__lte=max_price)

# Для ManyToMany потрібно distinct, щоб не втрачати товари
products = products.distinct()

context = {
    'products': products,
    'categories': categories,
    'colors': colors,
    'sizes': sizes,
}

return render(request, 'products/product_list.html', context)

```

Додаток 3

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">
  <title>Список товарів</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.1.3/dist/css/bootstrap.min.css"
integrity="sha384-MCw98/SFnGE8fJT3GXwEOngsV7Zt27NXFoaoApmYm81iuXoPkFOJwJ8ERdknLPMO"
crossorigin="anonymous">
</head>
<body class="flex-container" style="margin-left: 25px;">
  <nav class="navbar navbar-light bg-light">
    <div class="form-inline" style="display: flex; flex-direction: row;
align-content: space-between;">
      <button class="btn btn-primary" style="margin-right: 25px;">Додати
товар</button>
      <!-- Кнопка фільтру -->
      <button class="btn btn-secondary" id="filterToggle">
        Фільтрувати товари
      </button>

      <!-- Бокове меню фільтру -->
      <div id="filterSidebar" class="position-fixed top-0 end-0 bg-light
border-start p-3"
        style="width: 300px; height: 100%; display: none; z-index: 1050;
right: 0; top: 0; overflow-y: auto;">
        <h5>Фільтри</h5>
        <button type="button" class="close float-end mb-3"
id="closeFilter"></button>

        <form method="GET" action="{% url 'product_list' %}">
          <!-- Категорії -->
          <div class="mb-3">
            <label class="form-label">Категорія</label>
            {% for cat in categories %}
              <div class="form-check">
                <input class="form-check-input" type="checkbox"
name="category" value="{{ cat.id }}" id="cat{{ cat.id }}">
```

```

        <label class="form-check-label" for="cat{{ cat.id
}}">{{ cat.category }}</label>
    </div>
    {% endfor %}
</div>

<!-- Кольори -->
<div class="mb-3">
    <label class="form-label">Колір</label>
    {% for c in colors %}
        <div class="form-check">
            <input class="form-check-input" type="checkbox"
name="color" value="{{ c.id }}" id="color{{ c.id }}">
            <label class="form-check-label" for="color{{ c.id
}}">{{ c.color }}</label>
        </div>
    {% endfor %}
</div>

<!-- Розміри -->
<div class="mb-3">
    <label class="form-label">Розмір</label>
    {% for s in sizes %}
        <div class="form-check">
            <input class="form-check-input" type="checkbox"
name="size" value="{{ s.id }}" id="size{{ s.id }}">
            <label class="form-check-label" for="size{{ s.id
}}">{{ s.size }}</label>
        </div>
    {% endfor %}
</div>

<!-- Ціна -->
<div class="mb-3">
    <label class="form-label">Ціна від</label>
    <input type="number" class="form-control mb-1"
name="min_price" placeholder="0">
    <label class="form-label">Ціна до</label>
    <input type="number" class="form-control" name="max_price"
placeholder="1000">
</div>

```

```

        <button type="submit" class="btn btn-primary w-100
mt-2">Застосувати</button>
    </form>
</div>
<h1>Список товарів</h1>
<a href="{% url 'export_excel' %}">
    <button class="btn btn-info" style="margin-left: 25px;">Експортувати
в Excel</button>
</a>
</div>
</nav>
<div class="container">
    <div class="row">
        {% for product in products %}
            <div class="col-12 col-sm-6 col-md-3 mb-4"> <!-- 12 колонок / 3 = 4
картки -->
                <div class="card h-100">
                    {% if product.image %}
                        
                    {% else %}
                        <div class="card-img-top text-center p-5">Немає фото</div>
                    {% endif %}

                    <div class="card-body">
                        <h5 class="card-title">{{ product.code }}</h5>
                        <p class="card-text">{{ product.name }}</p>
                        <p class="card-text">{{ product.price }} UAH</p>
                    </div>
                </div>
            </div>
        {% endfor %}
    </div>
</div>

<br>

<script src="https://code.jquery.com/jquery-3.3.1.slim.min.js"
integrity="sha384-q8i/X+965DzO0rT7abK41JStQIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo"
crossorigin="anonymous"></script>

```



```

<script
src="https://cdn.jsdelivr.net/npm/popper.js@1.14.3/dist/umd/popper.min.js"
integrity="sha384-ZMP7rVo3mIykV+2+9J3UJ46jBk0WLaUAdn689aCwoqbBJiSnjAK/l8WvCWPIpM49"
crossorigin="anonymous"></script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.1.3/dist/js/bootstrap.min.js"
integrity="sha384-ChfqquxZUCnJSK3+MXmPNIyE6ZbWh2IMqE24lrYiqJxyMiZ6OW/JmZQ5stwEULTy"
crossorigin="anonymous"></script>

<script>
const filterToggle = document.getElementById('filterToggle');
const filterSidebar = document.getElementById('filterSidebar');
const closeFilter = document.getElementById('closeFilter');

filterToggle.addEventListener('click', () => {
    filterSidebar.style.display = 'block';
});

closeFilter.addEventListener('click', () => {
    filterSidebar.style.display = 'none';
});

// закрити по кліку поза меню
document.addEventListener('click', (e) => {
    if (!filterSidebar.contains(e.target) && e.target !== filterToggle) {
        filterSidebar.style.display = 'none';
    }
});
</script>
</body>
</html>

```